CSCE 46103/56103 Artificial Intelligence – Fall 2026

Backpropagation and Optimization

Jungtaek Kim
jungtaek.kim@uark.edu

University of Arkansas

November 3, 2026

UNIVERSITY OF ARKANSAS

This Material's Topics

- Connecting a training loss to a weight update.
- Applying the chain rule through a computation graph.
- Computing output-layer and hidden-layer gradients.
- Training with learning rates and minibatches.

The Central Question

How do we change the weights to improve the network's predictions?

Learning algorithms: Russell and Norvig [2020, Sec. 21.4].

UNIVERSITY OF ARKANSAS

Gradients and Weight Updates

Collect all weights and biases into $\mathbf{w}$. Let $J(\mathbf{w})$ be the training objective.

$$\nabla_{\mathbf{w}} J = \begin{bmatrix} \partial J/\partial w_1 & \cdots & \partial J/\partial w_p \end{bmatrix}^{\mathsf{T}}.$$

Each component measures how the objective changes when one parameter changes.

Learning algorithms: Russell and Norvig [2020, Sec. 21.4].

Gradients and Weight Updates

Collect all weights and biases into $\mathbf{w}$. Let $J(\mathbf{w})$ be the training objective.

$$\nabla_{\mathbf{w}} J = \begin{bmatrix} \partial J/\partial w_1 & \cdots & \partial J/\partial w_p \end{bmatrix}^{\mathsf{T}}.$$

Each component measures how the objective changes when one parameter changes.

Gradient Descent

For a learning rate $\alpha > 0$,

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} J.$$

A positive derivative decreases its weight; a negative derivative increases it.

Learning algorithms: Russell and Norvig [2020, Sec. 21.4].

Example Network

Nodes $1, 2$ are inputs, $3, 4$ are hidden neurons, and 5 is the output.
Let $w_{i,j}$ be the weight from node i to neuron j; $w_{0,j}$ is neuron j's bias.
Use $\mathbf{x} = (1, 2)^\mathsf{T}$, target $y = 2$, and ReLU hidden neurons.

Hidden Layer

$$z_3 = w_{0,3} + w_{1,3}x_1 + w_{2,3}x_2,$$
$$z_4 = w_{0,4} + w_{1,4}x_1 + w_{2,4}x_2,$$
$$a_3 = \max(0, z_3), \quad a_4 = \max(0, z_4).$$

Initially,

$$(w_{0,3}, w_{1,3}, w_{2,3}) = (0, 1, 0),$$
$$(w_{0,4}, w_{1,4}, w_{2,4}) = (0, 0, 1).$$

Linear Output and Squared Loss

$$\hat{y} = w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4.$$

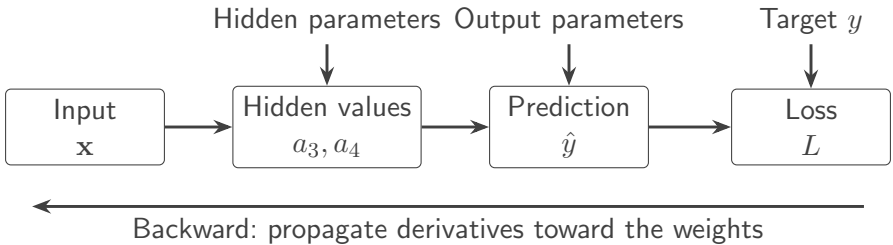
Initially,

$$(w_{0,5}, w_{3,5}, w_{4,5}) = (0, 0.5, 0.5).$$

$$a_3 = 1, \quad a_4 = 2, \quad \hat{y} = 1.5,$$
$$L = (\hat{y} - y)^2 = 0.25.$$

Network architecture: Russell and Norvig [2020, Fig. 21.3]. Weights, input, target, and learning rate are classroom examples.

Forward and Backward Passes



- The forward pass uses the input and current weights to compute the loss.
- Store intermediate values such as z_3, z_4, a_3, a_4, and $\hat{y}$.
- The backward pass reuses these values to compute $\partial L/\partial w$ for every parameter.

Computation graphs and reverse-mode differentiation: Russell and Norvig [2020, Sec. 21.4.1].

The Chain Rule

If $a = g(z)$ and $L = \ell(a)$, changing z changes L through a:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}.$$

Weighted Sum

For $z = b + w_1 x_1 + w_2 x_2$,

$$\frac{\partial z}{\partial w_1} = x_1, \qquad \frac{\partial z}{\partial b} = 1.$$

ReLU

For $a = \max(0, z)$,

$$\frac{\partial a}{\partial z} = \begin{cases} 1, & z > 0, \\ 0, & z < 0. \end{cases}$$

Both hidden inputs in our example are positive.

Gradients and backpropagation: Russell and Norvig [2020, Sec. 21.1.2].

Output-Layer Gradients

Start at the loss and move toward the weights:

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = 2(1.5 - 2) = -1.$$

Since $\hat{y} = w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4$,

$$\frac{\partial L}{\partial w_{3,5}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_{3,5}} = (-1)a_3 = -1.$$

Network architecture: Russell and Norvig [2020, Fig. 21.3]. Weights, input, target, and learning rate are classroom examples.

Output-Layer Gradients

Start at the loss and move toward the weights:

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = 2(1.5 - 2) = -1.$$

Since $\hat{y} = w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4$,

$$\frac{\partial L}{\partial w_{3,5}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_{3,5}} = (-1)a_3 = -1.$$

Parameter	$w_{0,5}$	$w_{3,5}$	$w_{4,5}$
Loss derivative	-1	-1	-2

Network architecture: Russell and Norvig [2020, Fig. 21.3]. Weights, input, target, and learning rate are classroom examples.

A Hidden-Layer Gradient

The path from $w_{1,3}$ to the loss is

$$w_{1,3} \longrightarrow z_3 \longrightarrow a_3 \longrightarrow \hat{y} \longrightarrow L.$$

Apply the chain rule along the path:

$$\frac{\partial L}{\partial w_{1,3}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_3} \frac{\partial a_3}{\partial z_3} \frac{\partial z_3}{\partial w_{1,3}}$$

$$= (-1) \times 0.5 \times 1 \times 1 = -0.5.$$

Network architecture: Russell and Norvig [2020, Fig. 21.3]. Weights, input, target, and learning rate are classroom examples.

A Hidden-Layer Gradient

The path from $w_{1,3}$ to the loss is

$$w_{1,3} \longrightarrow z_3 \longrightarrow a_3 \longrightarrow \hat{y} \longrightarrow L.$$

Apply the chain rule along the path:

$$\frac{\partial L}{\partial w_{1,3}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_3} \frac{\partial a_3}{\partial z_3} \frac{\partial z_3}{\partial w_{1,3}}$$

$$= (-1) \times 0.5 \times 1 \times 1 = -0.5.$$

A Reusable Intermediate Derivative

$\delta_3 = \partial L / \partial z_3 = -0.5$, so

$$\frac{\partial L}{\partial w_{0,3}} = \delta_3, \quad \frac{\partial L}{\partial w_{1,3}} = \delta_3 x_1, \quad \frac{\partial L}{\partial w_{2,3}} = \delta_3 x_2.$$

Network architecture: Russell and Norvig [2020, Fig. 21.3]. Weights, input, target, and learning rate are classroom examples.

Checkpoint: The Second Hidden Neuron

For neuron 4, $z_4 = 2$, $w_{4,5} = 0.5$, and $x_2 = 2$. We already know that $\partial L/\partial \hat{y} = -1$.

What are $\delta_4 = \partial L/\partial z_4$ and $\partial L/\partial w_{2,4}$?

Network architecture: Russell and Norvig [2020, Fig. 21.3]. Weights, input, target, and learning rate are classroom examples.

Checkpoint: The Second Hidden Neuron

For neuron 4, $z_4 = 2$, $w_{4,5} = 0.5$, and $x_2 = 2$. We already know that $\partial L/\partial \hat{y} = -1$.

What are $\delta_4 = \partial L/\partial z_4$ and $\partial L/\partial w_{2,4}$?

Answer

$$\delta_4 = (-1) \times 0.5 \times 1 = -0.5,$$

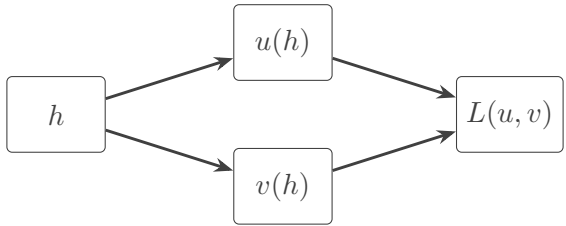
$$\frac{\partial L}{\partial w_{2,4}} = \delta_4 x_2 = (-0.5) \times 2 = -1.$$

The same chain-rule calculation applies to each hidden neuron.

Network architecture: Russell and Norvig [2020, Fig. 21.3]. Weights, input, target, and learning rate are classroom examples.

Gradients Through Branches

A value can affect the loss through more than one path.



$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial u}\frac{\partial u}{\partial h} + \frac{\partial L}{\partial v}\frac{\partial v}{\partial h}.$$

At each node, add the gradient contributions from its outgoing paths.

Computation graphs and reverse-mode differentiation: Russell and Norvig [2020, Sec. 21.4.1].

The Complete Gradient

All derivatives below use the same forward pass and original parameter values.

Destination	Parameters	Loss derivatives
Hidden neuron 3	$(w_{0,3}, w_{1,3}, w_{2,3})$	$(-0.5, -0.5, -1)$
Hidden neuron 4	$(w_{0,4}, w_{1,4}, w_{2,4})$	$(-0.5, -0.5, -1)$
Output neuron 5	$(w_{0,5}, w_{3,5}, w_{4,5})$	$(-1, -1, -2)$

Automatic Differentiation

A software library performs these local derivative calculations and accumulates their contributions through the graph.

Computation graphs and reverse-mode differentiation: Russell and Norvig [2020, Sec. 21.4.1].

One Gradient-Descent Step

Use $\alpha = 0.05$ and update every parameter with its computed gradient:

$$w^{\text{new}} = w - 0.05 \frac{\partial L}{\partial w}.$$

Parameters	Before	After
$(w_{0,3}, w_{1,3}, w_{2,3})$	$(0, 1, 0)$	$(0.025, 1.025, 0.05)$
$(w_{0,4}, w_{1,4}, w_{2,4})$	$(0, 0, 1)$	$(0.025, 0.025, 1.05)$
$(w_{0,5}, w_{3,5}, w_{4,5})$	$(0, 0.5, 0.5)$	$(0.05, 0.55, 0.60)$

For example, $w_{3,5}^{\text{new}} = 0.5 - 0.05(-1) = 0.55$.

Network architecture: Russell and Norvig [2020, Fig. 21.3]. Weights, input, target, and learning rate are classroom examples.

The Prediction After the Update

Run the forward pass again with the new weights and the same $\mathbf{x} = (1, 2)^{\mathsf{T}}$:

$$a_3^{\text{new}} = \max(0, 0.025 + 1.025 \times 1 + 0.05 \times 2) = 1.15,$$
$$a_4^{\text{new}} = \max(0, 0.025 + 0.025 \times 1 + 1.05 \times 2) = 2.15,$$
$$\hat{y}^{\text{new}} = 0.05 + 0.55 \times 1.15 + 0.60 \times 2.15 = 1.9725.$$

Network architecture: Russell and Norvig [2020, Fig. 21.3]. Weights, input, target, and learning rate are classroom examples.

The Prediction After the Update

Run the forward pass again with the new weights and the same $\mathbf{x} = (1, 2)^{\mathsf{T}}$:

$$a_3^{\text{new}} = \max(0, 0.025 + 1.025 \times 1 + 0.05 \times 2) = 1.15,$$
$$a_4^{\text{new}} = \max(0, 0.025 + 0.025 \times 1 + 1.05 \times 2) = 2.15,$$
$$\hat{y}^{\text{new}} = 0.05 + 0.55 \times 1.15 + 0.60 \times 2.15 = 1.9725.$$

The New Squared Loss

$$L^{\text{new}} = (1.9725 - 2)^2 = 0.00075625.$$

The loss on this training example decreased from 0.25.

Network architecture: Russell and Norvig [2020, Fig. 21.3]. Weights, input, target, and learning rate are classroom examples.

The Learning Rate

Consider a separate one-parameter objective $J(w) = (w-2)^2$. At $w = 1$, the derivative is $J'(1) = -2$ and the loss is $J(1) = 1$.

$$w^{\text{new}} = 1 - \alpha(-2) = 1 + 2\alpha.$$

Learning rate	New weight	New loss	Effect
0.1	1.2	0.64	Smaller loss
0.5	2.0	0	Minimum reached
1.1	3.2	1.44	Step too large

The learning rate controls the step size. Its useful scale depends on the objective.

Gradient descent: Russell and Norvig [2020, Sec. 21.4]. The quadratic is a classroom example.

Full Batches and Minibatches

For a training set of N examples, minimize the average loss:

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} L(h_{\mathbf{w}}(\mathbf{x}_i), y_i).$$

Update	Examples per step	Gradient used
Full batch	N	Average over all training examples
One-example SGD	1	Gradient from one sampled example
Minibatch SGD	m	Average over a sampled group

For a minibatch B of size m,

$$\mathbf{g}_B = \frac{1}{m} \sum_{i \in B} \nabla_{\mathbf{w}} L(h_{\mathbf{w}}(\mathbf{x}_i), y_i), \qquad \mathbf{w} \leftarrow \mathbf{w} - \alpha \mathbf{g}_B.$$

Learning algorithms: Russell and Norvig [2020, Sec. 21.4].

The Training Loop

Initialize the weights, then repeat over minibatches:

1. Select a minibatch of inputs and targets.
2. Run the forward pass and compute the average loss.
3. Run backpropagation to compute the loss gradients.
4. Apply the optimizer's parameter update.

Learning algorithms: Russell and Norvig [2020, Sec. 21.4].

The Training Loop

Initialize the weights, then repeat over minibatches:

1. Select a minibatch of inputs and targets.
2. Run the forward pass and compute the average loss.
3. Run backpropagation to compute the loss gradients.
4. Apply the optimizer's parameter update.

Steps and Epochs

A step is one parameter update. An epoch is one pass through the training set. Use validation performance to guide model selection and stopping.

Learning algorithms: Russell and Norvig [2020, Sec. 21.4].

Main Takeaways

- A forward pass computes predictions, intermediate values, and loss.
- Backpropagation applies the chain rule and sums contributions from multiple paths.
- The resulting gradients specify how each parameter affects the loss.
- Gradient descent uses these gradients and a learning rate to update the parameters.
- Minibatch SGD updates parameters using the average gradient of a sampled group.

Learning algorithms: Russell and Norvig [2020, Sec. 21.4].

Thank You

Any questions?

References

S. J. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020. URL `https://aima.cs.berkeley.edu/`.